

《数据库系统原理》报告

数据库系统原理作业 6

张鲋泮

北京交通大学计算机科学与技术学院 AI2201 班 22281052@bjtu.edu.cn

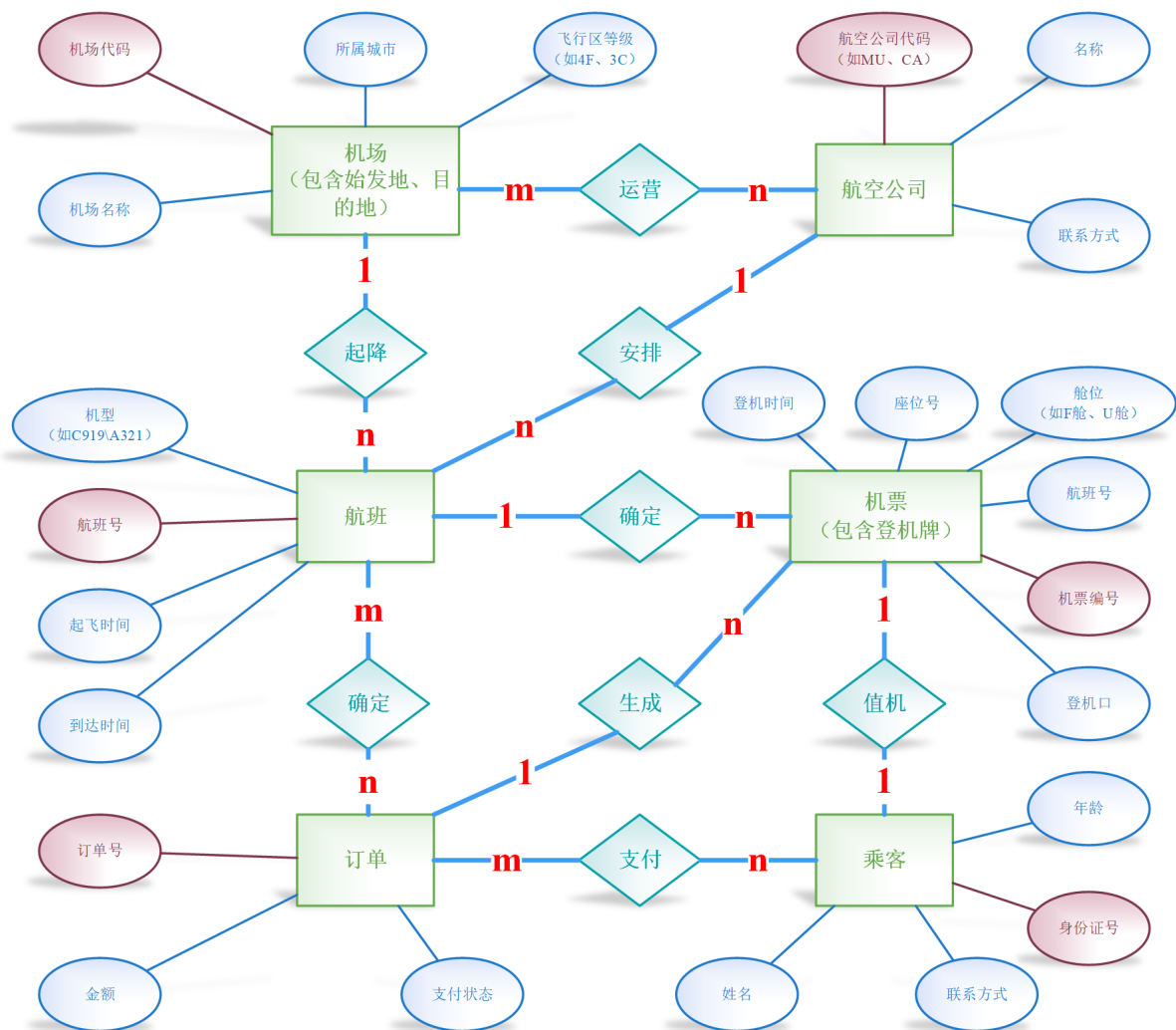


图 1: 航空公司航班查询业务模型 E-R 图, 来自于作业 2

1 题目 1: 数据库应用系统规划设计

题目要求: 针对前期作业中选定的业务背景, 请完成如下数据库应用系统规划训练

1. 给你所规划的数据库应用系统起一个系统名称
2. 尝试调研分析或自主规划设计该系统的业主企业或组织机构的组织架构图，并说明该系统涉及企业或组织机构中哪些相关业务部门。
3. 调研分析企业相关部门中的跟该系统有关的各种用户或企业外的用户，用文字描述用户使用系统开展业务的场景（例如，租车客户将在特定手机上打开 APP，用户在界面上点击…）。
4. 规划系统的性能指标，如并发用户数、用户数、核心业务响应时间等。
5. 说明你所规划的系统的战略地位，例如系统服务于公司以什么方式赢得客户、获得直接或间接收益，获得市场地位。
6. 尝试说明建设该系统可能涉及的投资和运营成本，分析可能获得的收益。
7. 结合系统业务功能与性能规划，确定初步的技术选型规划，大致分析该系统在技术上的可行性。

将以上内容形成系统规划与可行性分析报告，找两位同学当业务专家帮你论证方案的可行性，并在报告上列出专家姓名、专家论证建议和结论。

问题一部分，已经独立编写系统规划与可行性分析报告。详情请参见附件 1《航易行应用系统规划与可行性分析报告》。

2 题目 2：数据库应用系统需求分析

题目要求：自学统一建模语言 (*Unified Modeling Language, UML*)，在作业中解释 *UML*、用例 (*Use Case*)、用况 (*Use Case Scenario*)、用例图 (*Use Case Diagram*)、泳道图和数据流图 (*Data Flow Diagram, DFD*) 的概念及其 *VISIO* 中的画法，针对前述作业中选定的业务场景，开展如下系统需求分析：

1. 分析系统业务需求，标识出用户 (*Actor* 或 *User*)，进行系统功能划分，画出用例图；
2. 分析系统业务场景，针对涉及多个用户 (*Actor* 或用户) 的业务场景，分析业务流程，画出泳道图（至少一个）。
3. 结合前面作业设计结果，补充分析完善系统的数据项，数据结构（可以用类图表示），数据的存储（持久化）需求，形成数据字典。
4. 分析系统的数据处理需求，画出系统主要的数据流图，建议包含两个层级的 *DFD*。
5. 分析系统的非功能性需求，给出系统业务处理性能、安全性、完整性等需求。

将以上内容形成系统的需求规格说明书。

2.1 UML

UML 是一种用于软件及其他系统可视化、规范、构建和文档化的通用可视化建模语言。它提供了一套标准的图形符号和建模技术，帮助开发者、业务分析师和利益相关者在软件开发生命周期的各个阶段进行交流和理解。**UML 不是一种编程语言，而是一种图示语言**，用于描述系统的结构和行为。

用例 (Use Case) 用例描述了**系统外部的参与者 (Actor)** 如何使用系统来完成一个特定的业务目标。一个用例通常由系统和参与者之间的一系列交互（步骤）组成，这些交互最终为参与者带来了可测量的价值。用例捕获了系统的**功能性需求**，是从用户视角出发的功能说明。

用况 (Use Case Scenario) 用况是用例的一个**特定实例**或一条**执行路径**。它详细描述了在特定条件下，参与者和系统之间发生的一系列事件。一个用例可以有多个用况，代表了不同的成功路径、替代路径或异常路径。例如，“乘客查询航班”用例可能包括“成功查询并显示航班列表”的正常用况，以及“查询无结果”、“输入无效日期”等替代或异常用况。用况比用例更具体，有助于理解详细的交互流程和边界情况。

用例图 (Use Case Diagram) 用例图是 UML 中的一种图形，用于展示**系统、系统外部的参与者以及参与者与用例之间的关系**。它主要用于捕获系统的功能性需求，从外部视角描绘系统提供的各项功能。用例图通常包含：

- **系统边界 (System Boundary)**: 一个矩形框，代表所建模的系统。
- **用例 (Use Case)**: 用椭圆表示，位于系统边界内部，代表系统提供的具体功能。
- **参与者 (Actor)**: 用火柴人图形表示，位于系统边界外部，代表与系统交互的外部实体（人、其他系统或设备）。
- **关系 (Relationships)**:
 - **关联 (Association)**: 参与者与用例之间的连线，表示参与者与用例有交互。
 - **包含 (Include)**: 用例 A 包含用例 B，表示用例 A 的执行过程中必须包含用例 B 的功能。用带箭头的虚线表示，箭头指向被包含的用例。常用于提取用例中的公共部分。
 - **扩展 (Extend)**: 用例 A 扩展用例 B，表示在特定条件下，用例 A 的功能会可选地增加到用例 B 中。用带箭头的虚线和“«extend»”字样表示，箭头指向被扩展的用例。常用于描述用例的替代或异常流程。
 - **泛化 (Generalization)**: 参与者或用例之间的继承关系。

Visio 中的画法如下：

1. 选择“软件”模板类别，然后选择“UML 用例图”模板，点击“创建”。
2. 在左侧的“UML 用例”模具中，可以看到“参与者”、“用例”、“系统”等图形形状。
3. 将“系统”形状拖到绘图页面，调整大小以框住所有用例。在矩形上方可以添加系统名称。
4. 将“参与者”形状拖到系统边界外部，并为其命名。
5. 将“用例”形状拖到系统边界内部，并为其命名。
6. 使用“关联”、“包含”、“扩展”等连接线工具（通常在顶部工具栏的“连接线”类别中）连接相应的形状，表示关系。为包含和扩展关系添加对应的构造型（«include» 或 «extend»）。

泳道图 (Swimlane Diagram) 泳道图是一种特殊类型的活动图或流程图，用于展示业务流程中不同参与者（角色、组织单位或系统）负责执行哪些活动。泳道图将流程中的活动按照参与者进行分组，每个参与者对应一个“泳道”。泳道图清晰地展示了跨部门或跨角色的协作流程，有助于识别职责、沟通边界和潜在瓶颈。泳道图通常包含：

- **池 (Pool):** 代表整个业务流程或组织。
- **泳道 (Lane):** 位于池内部，代表流程中的一个具体参与者（如“外部乘客”、“客服人员”、“航易行系统”）。同一泳道内的活动由该参与者负责。
- **活动 (Activity):** 代表流程中的一个具体步骤或任务，位于相应的泳道内。
- **转换 (Transition):** 用带箭头的线连接活动，表示流程的流转顺序。
- **开始/结束节点 (Start/End Nodes):** 表示流程的起点和终点。

Visio 中的画法如下：

1. 选择“流程图”模板类别，然后选择“跨职能流程图”模板，点击“创建”。
2. 选择横向或纵向泳道。将“泳道”形状拖到绘图页面，调整大小。可以添加多个泳道并命名。
3. 在左侧的“基本流程图形状”等模具中，选择“流程”形状（代表活动）拖到相应的泳道内，并为其命名。
4. 使用“连接线”工具连接活动形状，表示流程步骤。
5. 添加开始/结束节点（通常在“基本流程图形状”模具中）。
6. 添加文本标签、注释和决策点（菱形）以丰富流程描述。

数据流图 (Data Flow Diagram, DFD) 数据流图 (DFD) 是一种图形技术，用于展示系统中数据如何在不同进程之间流动、如何存储以及如何与外部实体进行交互。它关注的是数据的流和转换，而不是控制流或处理顺序。DFD 是结构化分析方法的核心工具之一，有助于理解系统的逻辑功能和数据依赖关系。DFD 的主要组成元素包括：

- **进程 (Process)**: 用圆形或圆角矩形表示，代表对数据进行处理或转换的功能单元。
- **数据存储 (Data Store)**: 用平行线表示，代表系统中数据的存储位置（如数据库、文件）。
- **外部实体 (External Entity)**: 用矩形表示，代表系统外部的源或宿，是数据的起点或终点（如用户、其他系统、部门）。
- **数据流 (Data Flow)**: 用带箭头的线表示，代表数据在元素之间的流动方向。线上通常标注流动数据的名称。

DFD 可以分为不同层级（例如图2），以逐步细化系统功能：

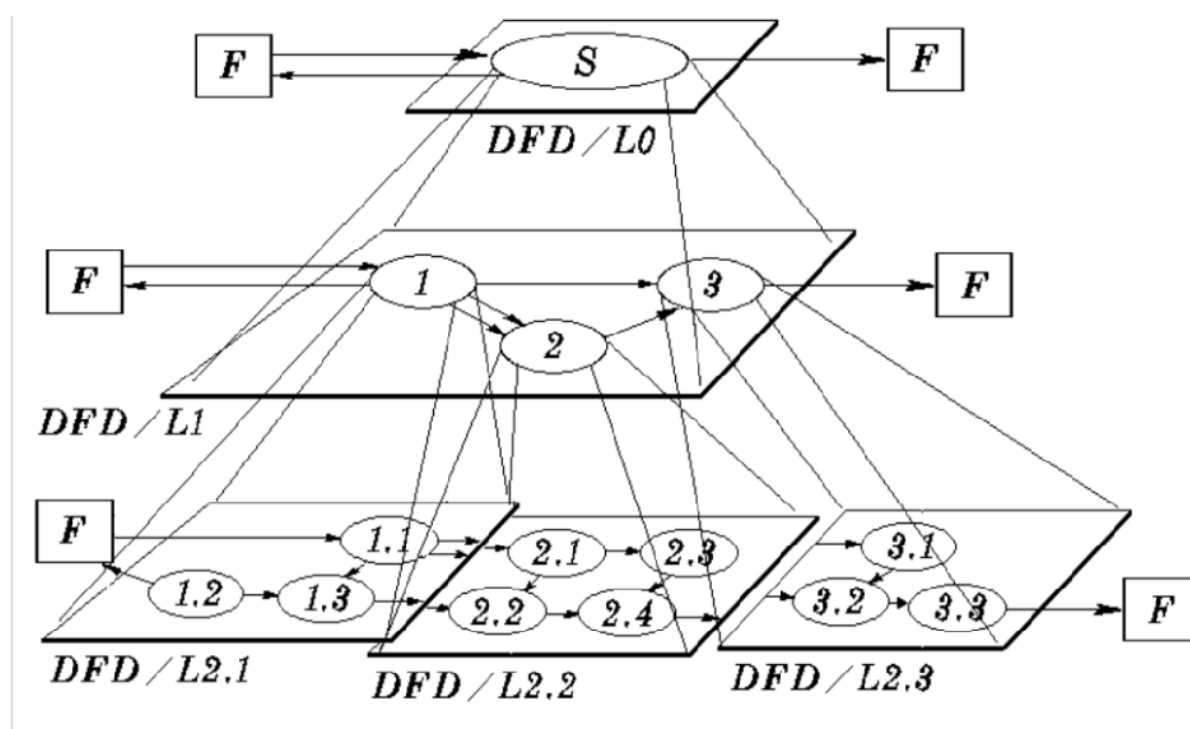


图 2: DFD 层级图

- **0 级 DFD (Context Diagram)**: 最高层级的 DFD，将整个系统视为一个单独的进程，只显示系统与外部实体之间的数据流，描绘系统的边界和外部接口。
- **1 级 DFD**: 对 0 级 DFD 中的主进程进行分解，展示系统的主要子进程、数据存储以及它们之间的数据流和与外部实体的数据交互。
- **2 级 DFD 及更低层级**: 进一步分解 1 级 DFD 中的子进程，展示更详细的数据处理步骤。

Visio 中的画法如下：

1. 选择“流程图”模板类别，然后选择“数据流图”模板，点击“创建”。
2. 在左侧的“数据流图形状”模具中，可以看到“进程”、“数据存储”、“外部交互方”等图形形状。
3. 将这些形状拖到绘图页面，并为其命名。
4. 使用“数据流”连接线工具（通常在顶部工具栏的“连接线”类别中）连接形状，表示数据流方向，并为数据流添加名称标签。
5. 对于不同层级的 DFD，可以复制或新建页面，并在父子图之间建立超链接关系（在形状上右键选择“超链接”）。

2.2 系统需求分析（应用于“航易行”系统）

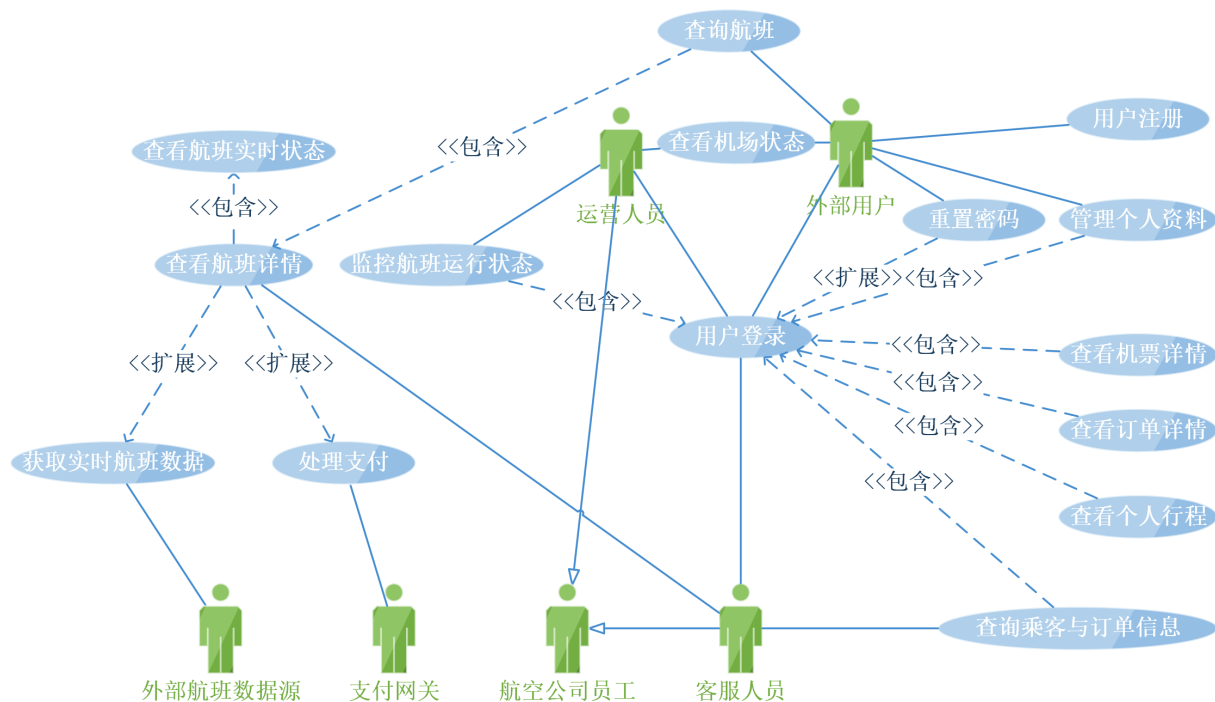
基于对航空公司航班查询业务的理解以及前期 UI 设计，对“航易行”系统进行如下需求分析。

2.2.1 业务需求与功能划分：用例图

此处用用例图展示了系统边界内的用例和系统外部的参与者，以及它们之间的关系。首先，系统外部参与者 (Actors) 如下：

- **外部乘客**：希望查询航班、管理个人行程、更新个人信息的个体用户。与系统进行直接互动，驱动核心航班信息和行程管理功能。
- **客服人员**：航空公司内部人员，需要查询详细的客户、订单和航班信息，以支持客户咨询和问题处理。与系统进行内部操作界面互动。
- **运营人员**：航空公司内部人员，需要监控航班实时状态、机场运行情况等，以支持运营调度。与系统进行内部监控界面互动。
- **支付网关**：第三方支付处理服务，系统需要与之交互完成支付流程。系统作为客户端向支付网关发送支付请求并接收支付结果通知。
- **外部航班数据源**：提供航班实时动态信息（如起飞/降落状态、延误信息）的外部系统（例如空管系统或其他数据聚合服务）。系统作为客户端接收或请求数据。

通过多个系统用例 (Use Cases)，可以构建一个关于业务需求与功能划分的用例图，如图3所示。



其中:

- **外部乘客** ◇ **用户注册**: 外部乘客通过该用例创建系统账户。
- **外部乘客** ◇ **用户登录**: 外部乘客通过该用例验证身份以访问系统。
- **外部乘客** ◇ **重置密码**: 外部乘客通过该用例恢复账户访问。
- **外部乘客** ◇ **查询航班**: 外部乘客通过该用例查找航班信息。
- **外部乘客** ◇ **查看机场状态**: 外部乘客通过该用例了解机场运行情况。
- **外部乘客** ◇ **管理个人资料**: 外部乘客通过该用例查看和修改个人信息。
- **客服人员** ◇ **用户登录**: 客服人员需要登录系统才能进行操作。
- **客服人员** ◇ **查询乘客与订单信息**: 客服人员通过该用例获取客户和交易详情。
- **客服人员** ◇ **查询航班详情**: 客服人员通过该用例获取特定航班详细信息。
- **运营人员** ◇ **用户登录**: 运营人员需要登录系统。
- **运营人员** ◇ **监控航班运行状态**: 运营人员通过该用例实时监控航班。
- **运营人员** ◇ **查看机场状态**: 运营人员通过该用例查看机场整体情况。
- **支付网关** ◇ **处理支付**: 支付网关与该用例（代表系统的支付处理功能）进行交互。

- **外部航班数据源 <> 获取实时航班数据**：外部数据源向该用例（代表系统的数据接收/请求功能）提供数据。
- **包含关系 (Include)**：用例 A 包含用例 B，表示用例 A 执行时必须调用用例 B 的功能。用带箭头的虚线表示，箭头从基用例（包含者）指向包含用例（被包含者）。
 - **管理个人资料 -<<include>>-> 用户登录**：用户必须登录后才能管理个人资料。
 - **查看个人行程 -<<include>>-> 用户登录**：用户必须登录后才能查看自己的行程。
 - **查看订单详情 -<<include>>-> 用户登录**：用户必须登录后才能查看订单详情。
 - **查看机票详情 -<<include>>-> 用户登录**：用户必须登录后才能查看机票详情。
 - **查询乘客与订单信息 -<<include>>-> 用户登录**：客服需要登录。
 - **监控航班运行状态 -<<include>>-> 用户登录**：运营需要登录。
 - **查询航班 -<<include>>-> 查看航班详情**：通常用户查询到航班列表后会进一步查看详情。
 - **查看航班详情 -<<include>>-> 查看航班实时状态**：在查看航班详情时，通常也会显示实时状态。
- **扩展关系 (Extend)**：用例 A 扩展用例 B，表示在特定条件下，用例 A 的功能会可选地增加到用例 B 中。用带箭头的虚线和“<<extend>>”字样表示，箭头从扩展用例 (A) 指向被扩展用例 (B)。
 - **重置密码 -<<extend>>-> 用户登录**：在用户登录失败或忘记密码时，可以选择执行重置密码功能。
 - **查看航班详情 -<<extend>>-> 处理支付**：（假设在详情页可直接购买）在查看航班详情后，乘客可以选择进入支付流程。
 - **查看航班详情 -<<extend>>-> 获取实时航班数据**：（假设详情页实时刷新状态）在查看详情时，系统可能会周期性获取实时数据。
- **泛化关系 (Generalization)**：表示继承关系。用带空心三角箭头的实线表示，箭头指向父类。
 - **客服人员 -|> 航空公司员工**（假设存在通用“航空公司员工”参与者）
 - **运营人员 -|> 航空公司员工**

2.2.2 业务场景分析：泳道图

泳道图又叫跨职能流程图，这种图清晰地展示了在特定业务流程中，不同参与者（泳道）之间活动的分配和流转。此处以**乘客通过客服查询航班状态**作为业务分析场景（见图4）。池（Pool）命名为“乘客通过客服查询航班状态流程”，包含三个泳道（Lanes）：“外部乘客”、“客服人员”和“航易行系统”。

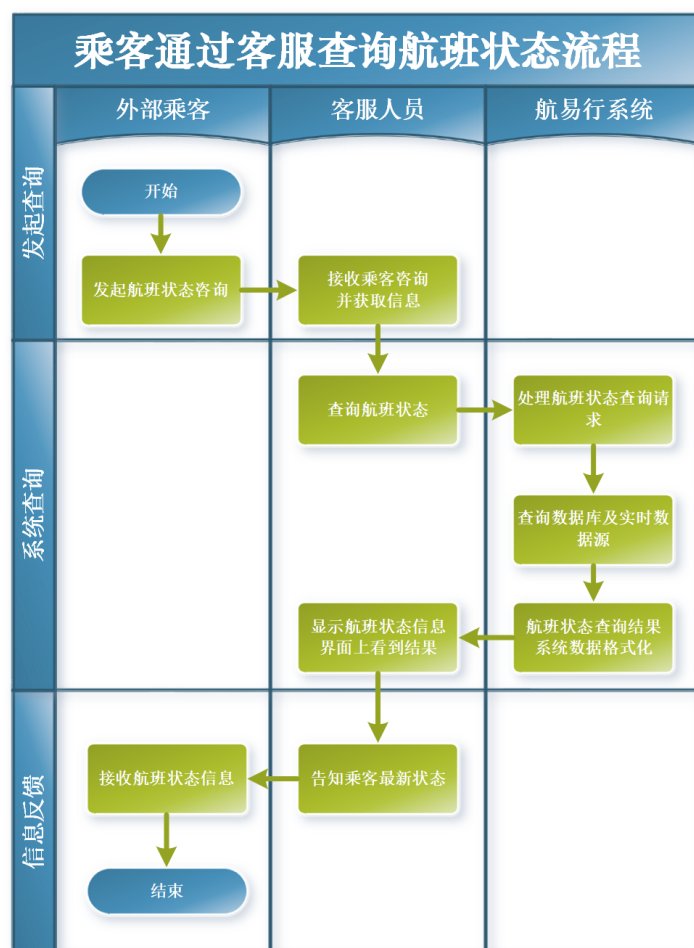


图 4: 业务场景乘客通过客服查询航班状态分析：泳道图

对于乘客通过客服查询航班状态，整个的流程如下：

1. 外部乘客泳道：活动“发起航班状态咨询”。
2. 转换：从“外部乘客”泳道中的“发起航班状态咨询”指向客服人员泳道中的“接收乘客咨询”。表示咨询信息从乘客传递给客服。
3. 客服人员泳道：活动“接收乘客咨询并获取信息”（如航班号、身份证号）。
4. 转换：从“客服人员”泳道中的“接收乘客咨询并获取信息”指向该泳道内的活动“查询航班状态”。表示客服人员根据获取的信息进行系统操作。
5. 客服人员泳道：活动“查询航班状态”。客服人员使用系统界面执行查询功能。
6. 转换：从“客服人员”泳道中的“查询航班状态”指向航易行系统泳道中的活动“处理航班状态查询请求”。表示客服人员通过界面向系统发送查询请求。
7. 航易行系统泳道：活动“处理航班状态查询请求”。系统接收请求。
8. 航易行系统泳道：活动“查询数据库及实时数据源”。系统内部活动，访问航班数据存储并可能调用实时数据集成功能。

9. 转换：从“查询数据库及实时数据源”指向该泳道内的活动“组织航班状态查询结果”。
10. 航易行系统泳道：活动“组织航班状态查询结果”。系统将数据格式化。
11. 转换：从“组织航班状态查询结果”指向客服人员泳道中的活动“显示航班状态信息”。表示系统将查询结果返回给客服工作台。
12. 客服人员泳道：活动“显示航班状态信息”。客服人员在界面上看到结果。
13. 转换：从“客服人员”泳道中的“显示航班状态信息”指向该泳道内的活动“告知乘客最新状态”。表示客服人员阅读并理解系统返回的信息。
14. 客服人员泳道：活动“告知乘客最新状态”。客服人员将信息传达给乘客。
15. 转换：从“告知乘客最新状态”指向外部乘客泳道中的活动“接收航班状态信息”。表示信息从客服传递给乘客。
16. 外部乘客泳道：活动“接收航班状态信息”。指向结束节点。流程结束。

2.2.3 数据项、数据结构、数据存储需求与数据字典

本系统的数据需求主要围绕航空公司航班查询业务的核心实体：机场、航空公司、航班、乘客、订单、机票。

数据项与数据结构 根据 ER 图和表结构设计，系统的核心数据结构可以基于以下实体及其属性来描述。这些实体可以视为面向对象分析中的“类”，它们之间的关系则构成了数据结构。

- 机场 (Airport)

属性：机场代码（主键）、机场名称、所属城市、飞行区等级。

- 航空公司 (Airline)

属性：公司代码（主键）、名称、联系方式。

- 航班 (Flight)

属性：航班号（主键）、机型、起飞时间、到达时间、出发机场代码（外键）、目的机场代码（外键）、航空公司代码（外键）。

- 乘客 (Passenger)

属性：身份证号（主键）、姓名、年龄、联系方式。

- 订单 (Orderinfo)

属性：订单号（主键）、乘客身份证号（外键）、总金额、支付状态。

- 机票 (Ticket)

属性： 机票编号（主键）、航班号（外键）、乘客身份证号（外键）、座位号、登机口、登机时间、舱位。

数据存储（持久化）需求 系统的所有核心业务数据都必须进行持久化存储，以确保数据的长期可用性和一致性。

- 可靠性：数据存储必须具备高可靠性，防止数据丢失或损坏。需要采用冗余存储（如 RAID）、定期备份和灾难恢复机制。
- 一致性：存储的数据必须保持内部一致性，符合业务规则和数据模型（通过数据库完整性约束实现）。
- 安全性：敏感数据（如乘客身份证号、联系方式、支付信息）在存储时需要考虑加密或脱敏处理。访问控制机制需要限制对数据的访问权限。
- 性能：存储系统需要支持高吞吐量的读写操作和快速的数据检索，以满足高并发查询的需求。
- 可扩展性：存储容量应能随着数据量的增长而灵活扩展。

数据字典 数据字典是对系统中所有数据项、数据结构、数据流和数据存储的集中式描述。它提供了数据项的标准化定义和属性信息，是理解和管理系统数据的重要工具。一个完整的数据字典将包含系统中所有数据元素的详细信息。以下表格是对于本数据库系统的数据字典。

表 1: 数据表: Airport (机场)

字段名称	描述	数据类型	可为空	键/约束	引用表
airport_code	唯一标识机场的代码	VARCHAR(10)	否	PRIMARY KEY	
airport_name	机场全称	VARCHAR(100)	否	NOT NULL	
city	机场所在城市	VARCHAR(50)	否	NOT NULL	
flight_zone	机场飞行区等级	VARCHAR(10)	否	NOT NULL	

表 2: 数据表: Airline (航空公司)

字段名称	描述	数据类型	可为空	键/约束	引用表
airline_code	唯一标识航空公司的代码	VARCHAR(10)	否	PRIMARY KEY	
airline_name	航空公司名称	VARCHAR(100)	否	NOT NULL	
contact_info	航空公司联系方式	VARCHAR(100)	是		

表 3: 数据表: Flight (航班)

字段名称	描述	数据类型	可为空	键/约束	引用表
flight_number	唯一标识航班的编号	VARCHAR(20)	否	PRIMARY KEY	
air_model	航班执飞的机型	VARCHAR(50)	否	NOT NULL	
departure_time	计划起飞日期和时间	DATETIME	否	NOT NULL	
arrival_time	计划到达日期和时间	DATETIME	否	NOT NULL	
departure_airport	出发机场代码	VARCHAR(10)	否	NOT NULL, FOREIGN KEY	Airport
arrival_airport	到达机场代码	VARCHAR(10)	否	NOT NULL, FOREIGN KEY	Airport
airline_code	执飞航空公司的代码	VARCHAR(10)	否	NOT NULL, FOREIGN KEY	Airline

表 4: 数据表: Passenger (乘客)

字段名称	描述	数据类型	可为空	键/约束	引用表
id_card	乘客身份证号码	VARCHAR(20)	否	PRIMARY KEY	
passname	乘客姓名	VARCHAR(50)	否	NOT NULL	
age	乘客年龄	INT	否	NOT NULL, CHECK (age ≥ 0 AND age < 120)	
phone	乘客联系电话	VARCHAR(100)	是		

表 5: 数据表: Ticket (机票)

字段名称	描述	数据类型	可为空	键/约束	引用表
ticket_id	唯一标识机票/登机牌的编号	INT	否	PRIMARY KEY, AUTO_INCREMENT	
flight_number	关联的航班编号	VARCHAR(20)	否	NOT NULL, FOREIGN KEY	Flight
id_card	关联的乘客身份证号	VARCHAR(20)	否	NOT NULL, FOREIGN KEY	Passenger
seat_number	分配的座位号	VARCHAR(10)	否	NOT NULL	
gate	分配的登机口	VARCHAR(10)	是		
boarding_time	计划登机日期和时间	DATETIME	否	NOT NULL	
cabin_seat	分配的舱位	VARCHAR(10)	是		

表 6: 数据表: Orderinfo (订单)

字段名称	描述	数据类型	可为空	键/约束	引用表
order_id	唯一标识订单的编号	INT	否	PRIMARY KEY, AUTO_INCREMENT	
id_card	下单乘客的身份证号	VARCHAR(20)	否	NOT NULL, FOREIGN KEY	Passenger
moneys	订单总金额	DECIMAL(10, 2)	否	NOT NULL	
payment	支付状态	INT	否	NOT NULL, CHECK (payment IN (0, 1))	

2.2.4 数据处理需求：数据流图（DFD）

对于航易行系统整个的数据处理需求，用 DFD 展示，如图5所示。

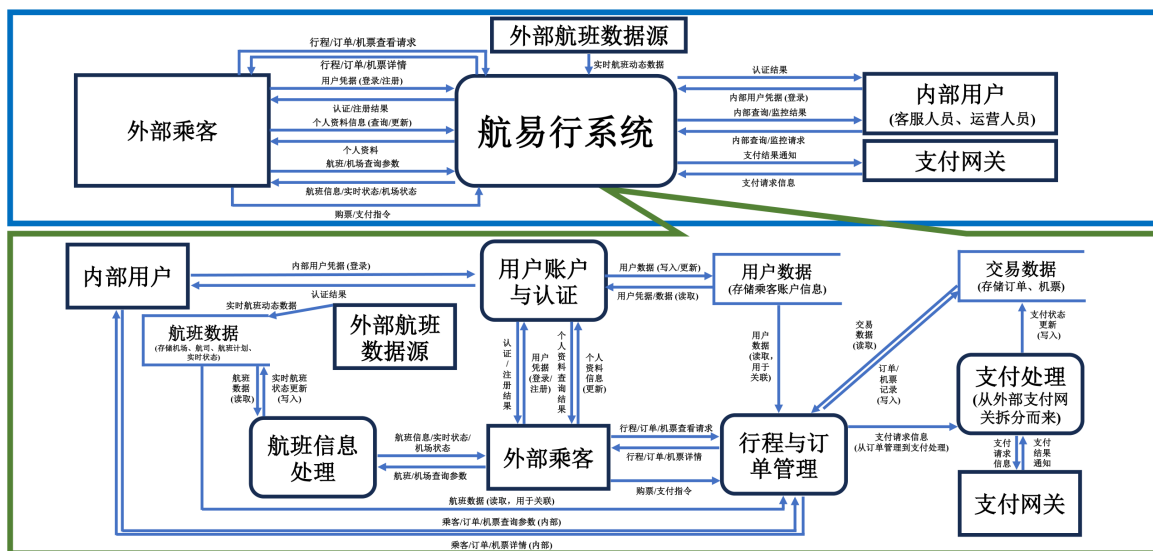


图 5: 数据处理需求：数据流图（DFD）

0 级 DFD 在 0 级 DFD 中，整个系统被抽象为一个中心进程，命名为“航易行系统”。该系统与外部实体之间通过数据流连接，表示数据的输入和输出。外部实体分别是外部乘客、内部用户（合并了客服人员、运营人员）、支付网关和外部航班数据源。

数据流的流向和内容如下：

- **外部乘客**：向“航易行系统”传递用户凭据（登录/注册）、个人资料信息（查询/更新）、航班/机场查询参数、行程/订单/机票查看请求以及购票/支付指令。并且可以从“航易行系统”接收认证/注册结果、个人资料、航班信息、实时航班状态、机场状态、行程/订单/机票详情。

- **内部用户：**向“航易行系统”传递内部用户凭据（登录）以及内部查询/监控请求（乘客、订单、航班、机场）。并且可以从“航易行系统”接收认证结果、乘客/订单/航班/状态的监控结果。
- **支付网关：**从“航易行系统”接收支付请求信息（订单号、金额、支付方等）。并且可以向“航易行系统”返回支付结果通知（支付成功/失败状态、交易号等）。
- **外部航班数据源：**向“航易行系统”提供实时航班动态数据（航班号、最新起降时间、状态、延误原因等）。

1 级 DFD：系统主要功能分解 在 1 级 DFD 中，“航易行系统”进程被细分为多个子进程，展示了系统内部的主要功能模块，并考虑了与外部实体的数据交互以及数据存储的关系。具体的进程如下：

- **P1 用户账户与认证：**该进程负责处理外部乘客和内部用户的登录/注册、个人资料信息的更新以及用户凭据的验证。
- **航班信息处理**该进程负责接收并处理来自外部乘客和内部用户的航班查询请求，提供航班信息、实时状态和机场状态。它还从外部航班数据源接收实时航班动态数据，并更新航班状态。
- **P2 行程与订单管理**该进程负责处理外部乘客的行程/订单/机票查看请求，生成订单记录并传递支付请求信息。它与数据存储（用户数据、航班数据、交易数据）进行交互，以提供详细的订单和行程信息。
- **P3 支付处理**从外部支付网关接收支付请求，并处理支付结果。该进程负责更新交易数据中的支付状态。

数据存储如下：

- **DS1 用户数据：**存储乘客的账户信息，包括用户凭据、个人资料等。
- **DS2 航班数据：**存储机场、航空公司、航班计划、实时航班状态等信息。
- **DS3 交易数据：**存储订单和机票信息，包括订单的支付状态等。

数据流及关系如下：

- **外部乘客与 P1：**
 - 外部乘客向 P1 发送用户凭据（登录/注册），P1 验证并处理，向外部乘客返回认证/注册结果。外部乘客还向 P1 发送个人资料信息更新，P1 更新 DS1 中的用户数据，并返回更新后的个人资料。
- **P2 与外部乘客：**外部乘客向 P2 发送航班/机场查询参数，P2 查询 DS2 中的航班数据，并向外部乘客返回航班信息、实时状态和机场状态。

- **外部航班数据源与 P2:** 外部航班数据源向 P2 提供实时航班动态数据, P2 处理这些数据并将实时航班状态更新到 DS2 中。
- **P3 与外部乘客:** 外部乘客向 P3 发送行程/订单/机票查看请求, P3 从 DS1、DS2 和 DS3 中获取相关数据, 并向外部乘客返回行程/订单/机票详情。外部乘客还会向 P3 发送购票/支付指令, P3 将订单记录写入 DS3, 并将支付请求信息传递给 P5。
- **P5 与支付网关:** P5 向支付网关发送支付请求信息, 支付网关处理后返回支付结果通知, P5 更新 DS3 中的支付状态。
- **内部用户与进程交互:** 内部用户与 P1 进行登录, 获取认证结果; 向 P2 发送航班/机场查询参数, 获取航班信息、实时状态和机场状态; 向 P3 发送乘客/订单/机票查询参数, 获取相关详情。

2.2.5 非功能性需求 (NFRs)

非功能性需求描述了系统应该具备的质量属性。以下是部分航易行系统的 NFRs:

- **业务处理性能:**
 - **响应时间:** 95% 的航班查询请求应在 200 毫秒内返回结果; 95% 的行程查看请求应在 100 毫秒内返回结果; 95% 的个人资料查询应在 50 毫秒内返回结果; 95% 的个人资料更新应在 500 毫秒内完成。
 - **吞吐量:** 系统应能支持每秒处理至少 5000 次查询请求, 每秒处理至少 100 次写入/更新操作。
 - **并发用户:** 系统应能支持至少 10000 名并发活跃用户。
 - **扩展性:** 系统应能通过增加硬件资源 (如服务器、存储容量) 或调整配置, 支持未来 5 年内用户量和数据量每年 20% 的增长, 并保持性能指标在可接受范围内。
- **安全性:**
 - **用户认证:** 支持基于手机号/邮箱和密码的身份验证。密码应使用安全的哈希算法 (如 Argon2 或 bcrypt) 加盐存储。
 - **用户授权:** 实现基于角色的访问控制 (RBAC), 不同类型的用户 (乘客、客服、运营等) 只能访问其被授权的功能和数据。对于敏感数据 (如乘客身份证号), 应实现列级别的访问控制。
 - **数据保密性:** 存储的敏感数据 (如身份证号、联系方式) 应进行加密或脱敏处理。数据在网络传输过程中应使用加密协议 (如 HTTPS)。
 - **数据完整性:** 防止未经授权的数据修改。通过数据库完整性约束 (主键、外键、唯一性、检查约束) 保证数据的逻辑一致性。

- 审计: 记录关键的安全相关事件, 如用户登录失败、敏感数据访问尝试、数据修改等。
- 防攻击: 具备基本的网络安全防护能力, 如防火墙、入侵检测、防 DDoS 攻击等。
- 完整性:
 - **实体完整性:** 系统应验证输入数据的合法性 (如年龄范围、日期格式、机场代码有效性), 键字段必须唯一, 且不能为空。
 - **参照完整性:** 保持关联数据之间的一致性 (通过外键约束)。例如, 机票记录必须关联一个有效的航班和一个有效的乘客。
 - **用户定义完整性:** 确保数据符合业务逻辑规则 (通过检查约束或业务逻辑代码), 例如航班到达时间必须晚于起飞时间, 订单支付状态只能是预设值。
- 可用性:
 - 系统应保证至少 99.95% 的年度可用性 (全年计划外停机时间少于 4.38 小时)。
 - 系统应具备故障转移能力, 当部分组件发生故障时, 系统能自动切换到备用组件, 不影响整体服务。
 - 系统应稳定运行, 减少故障频率。
- 可靠性:
 - 具备完善的错误处理和日志记录机制, 便于故障排查和恢复。
 - 关键数据 (如航班、订单、机票) 应定期备份, 并具备快速恢复能力 (RPO 和 RTO 符合业务需求)。
 - 系统架构清晰, 模块化程度高, 代码规范, 易于理解和修改。
 - 提供完善的部署、配置和监控文档。

2.3 航易行系统需求规格说明书

需求规格说明书部分, 已经独立编写需求规格说明书。详情请参见附件 2 《航易行应用系统需求规格说明书》。

3 题目 3: 数据库应用系统设计—概念与逻辑设计

基于以上需求规格说明书, 完成如下任务:

- 结合新补充的需求, 完善 ER 图, 形成完整的实体关系模型。
- 结合前面几次作业设计的数据模式, 在函数依赖的范畴判定原有模式的规范化程度, 并根据需要补充新关系模式。并对不属于 3NF 的模式进行分解, 使其达到 3NF 的要求。

以上内容形成第一版的系统设计规格说明，以后持续完善。

根据新补充要求，对于此系统完善了 ER 图，如图6所示。

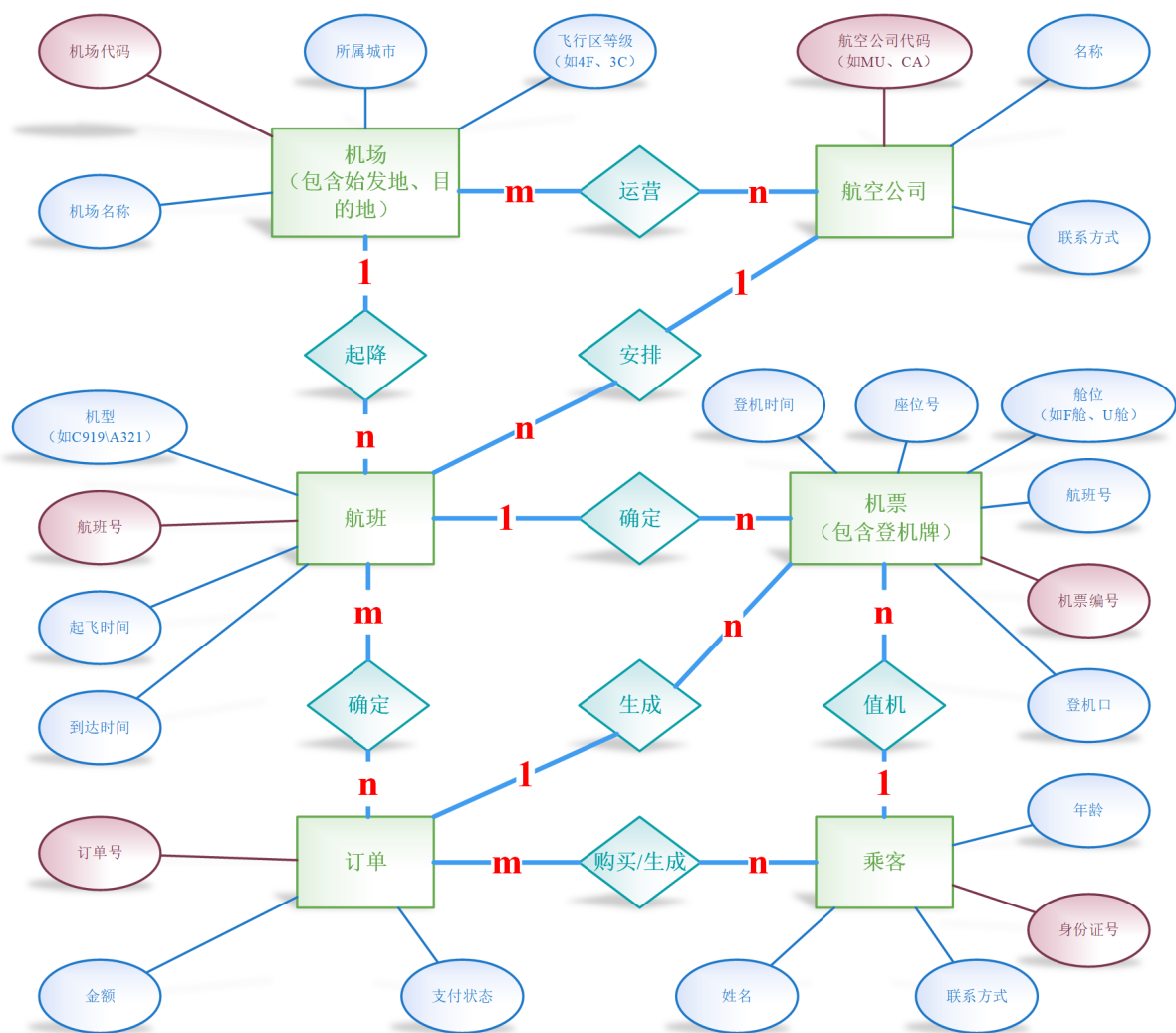


图 6: 完善后的航空公司航班查询业务模型 E-R 图

其中修改如下：

- 一个乘客拥/购买有多张机票，于是将乘客-机票关系改为了 1:n 关系（一对多关系）
- 乘客-订单关系修改为“购买/生成关系”

规范化是数据库设计理论，旨在通过分解关系模式来减少数据冗余，避免更新异常（插入、删除、修改异常）。对于课上学过的范式内容，这里进行一定的解释。

- **函数依赖 (FD):** 表示属性集合之间的一种约束关系，如果在一个关系中，属性集合 X 的值能唯一确定属性集合 Y 的值，则称 Y 函数依赖于 X，记作 $X \rightarrow Y$ 。
- **码 (Key):** 如果属性集合 K 能够函数确定关系中的所有属性，并且 K 的任何真子集都不能确定所有属性，则 K 是候选码。主键是选定的一个候选码。

- **1NF (第一范式):** 关系模式 R 中，每个属性的值域都是原子的，即不可再分。提供的 SQL DDL 中的所有列类型 (VARCHAR, INT, DATETIME, DECIMAL) 都表示原子值，符合 1NF。
- **2NF (第二范式):** 关系模式 R 属于 1NF，且每一个非主属性都完全函数依赖于 R 的每一个候选码。在以上模式中，所有模式的主键都是单个属性 (或单个属性构成)，因此不存在部分函数依赖于候选码的情况。所有模式都符合 2NF。
- **3NF (第三范式):** 关系模式 R 属于 2NF，且每一个非主属性都不传递依赖于 R 的每一个候选码。传递依赖是指 $X \rightarrow Y, Y \rightarrow Z$ ，其中 X 是码，Y 不是码，Z 是非主属性。

因此对目前的 6 个表进行分析：

- Airport (**airport_code**, airport_name, city, flight_zone)
 - 函数依赖: $\text{airport_code} \rightarrow \text{airport_name}, \text{city}, \text{flight_zone}$
 - 符合 1NF (属性原子性)。
 - airport_code 是单属性主键，因此所有非主属性都完全函数依赖于主键，符合 2NF。
 - 不存在非主属性之间的函数依赖或非主属性对主键的传递依赖，符合 3NF。
- Airline (**airline_code**, airline_name, contact_info)
 - 函数依赖: $\text{airline_code} \rightarrow \text{airline_name}, \text{contact_info}$
 - 符合 1NF。
 - airline_code 是单属性主键，所有非主属性完全函数依赖于主键，符合 2NF。
 - 不存在非主属性对主键的传递依赖，符合 3NF。
- Flight (**flight_number**, air_model, departure_time, arrival_time, departure_airport, arrival_airport, airline_code)
 - 函数依赖: $\text{flight_number} \rightarrow \text{air_model}, \text{departure_time}, \text{arrival_time}, \text{departure_airport}, \text{arrival_airport}, \text{airline_code}$
 - 符合 1NF。
 - flight_number 是单属性主键，所有非主属性完全函数依赖于主键，符合 2NF。
 - 不存在非主属性对主键的传递依赖 (airline_code $\rightarrow$ airline_name 是存在的，但 airline_code 本身就是主键决定的一部分，且 airline_name 不在这个表中，所以不构成对 flight_number 的传递依赖)，符合 3NF。
- Passenger (**id_card**, passname, age, phone)

- 函数依赖: id_card -> passname, age, phone
 - 符合 1NF。
 - id_card 是单属性主键，所有非主属性完全函数依赖于主键，符合 2NF。
 - 不存在非主属性对主键的传递依赖，符合 3NF。
- Ticket (**ticket_id**, flight_number, id_card, seat_number, gate, boarding_time, cabin_seat)
 - 函数依赖: ticket_id -> flight_number, id_card, seat_number, gate, boarding_time, cabin_seat
 - 符合 1NF。
 - ticket_id 是单属性主键，所有非主属性完全函数依赖于主键，符合 2NF。
 - 不存在非主属性对主键的传递依赖，符合 3NF。
- Orderinfo (**order_id**, id_card, moneys, payment)
 - 函数依赖: order_id -> id_card, moneys, payment
 - 符合 1NF。
 - order_id 是单属性主键，所有非主属性完全函数依赖于主键，符合 2NF。
 - 不存在非主属性对主键的传递依赖，符合 3NF。

检查代码后，目前的关系都符合 3NF。建立的 6 张表中，每一个非主属性都不传递依赖于 R 的每一个候选码，也就是说不存在传递依赖。因此对目前的表的构建不做相关的修改。

4 补充作业

完成教材（第六版 P.239）7,8 两题其中，各实体的属性如下（联系的属性根据需要添加）：

- 系：系编号，系名
- 班级：班级编号，班级名
- 教研室：教研室编号，教研室
- 学生：学号，姓名，学历
- 课程：课程编号，课程名
- 教员：职工号，姓名，职称
- 产品：产品号，产品名
- 零件：零件号，零件名

- 原材料：原材料号，原材料名，类别
- 仓库：仓库号，仓库名

将上题的 *E-R* 图转换成关系模型，指明每个关系模式的主键和外键，在函数依赖的范畴分析关系模式满足第几范式，并将不满足 *BCNF* 的关系模式分解成 *BCNF*。

4.1 题目 7

某学院有若干个系，每个系有若干班级和教研室，每个教研室有若干教师，其中有的教授和副教授每人各带若干研究生，每个班有若干学生，每个学生选修若干课程，每门课可由若干学生选修，某学生选修某一门课程有一个成绩。请用 *E-R* 图画出此应用场景的概念模型。

通过题目要求，画出了学校教务场景 *E-R* 图（如图7）。其中，此处省略了“学校”这个实体（因为确实没什么属性），认为教师与课程之间应该也有关联，于是添加了相关的联系（图7中的紫色所示）。

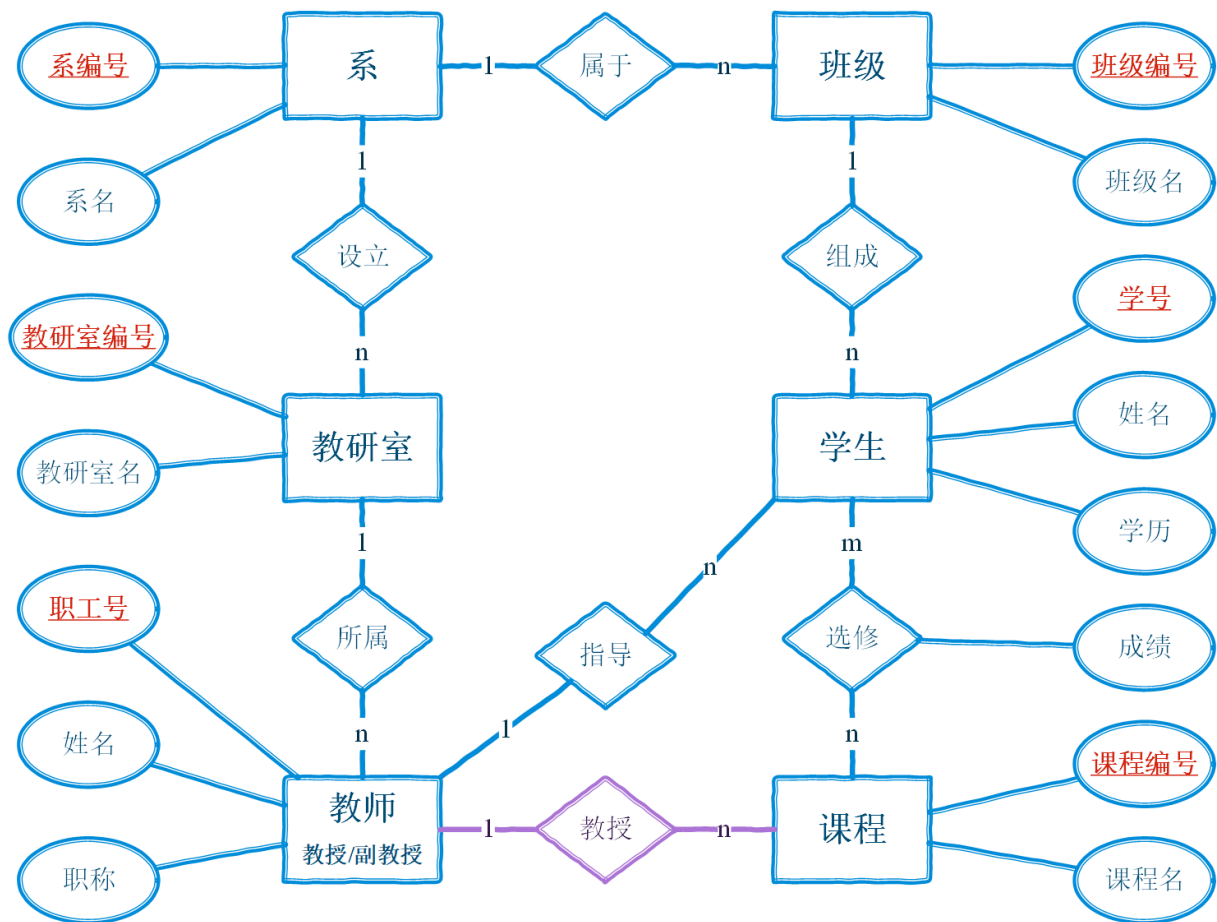


图 7: 学校教务场景 *E-R* 图

关系模型转换如下：

- **系** (系编号, 系名)
主键: 系编号

外键: 无

- **教研室** (教研室编号, 教研室名) 主键: 教研室编号

外键: 无

- **教师** (职工号, 姓名, 职称)

主键: 职工号

外键: 无

- **班级** (班级编号, 班级名)

键: 班级编号

外键: 无

- **学生** (学号, 姓名, 学历)

主键: 学号

外键: 无

- **课程** (课程编号, 课程名)

主键: 课程编号

外键: 无

- **选修关系** (学号, 课程编号, 成绩)

主键: (学号, 课程编号) (复合主键)

外键: 学号, 课程编号

在目前的情况下, 上述所有关系模式都已满足 BCNF, 无需进一步分解。

4.2 题目 8

某工厂生产若干产品, 每种产品由不同的零件组成, 有的零件可用在不同的产品上。这些零件由不同的原材料制成, 不同零件所用的材料可以相同。这些零件按所属的不同产品分别放在仓库中, 原材料按照类别放在若干仓库中。请用 *E-R* 图画出此工厂产品、零件、材料、仓库的概念模型。

通过题目要求, 画出了工厂生产场景 *E-R* 图 (如图8)。

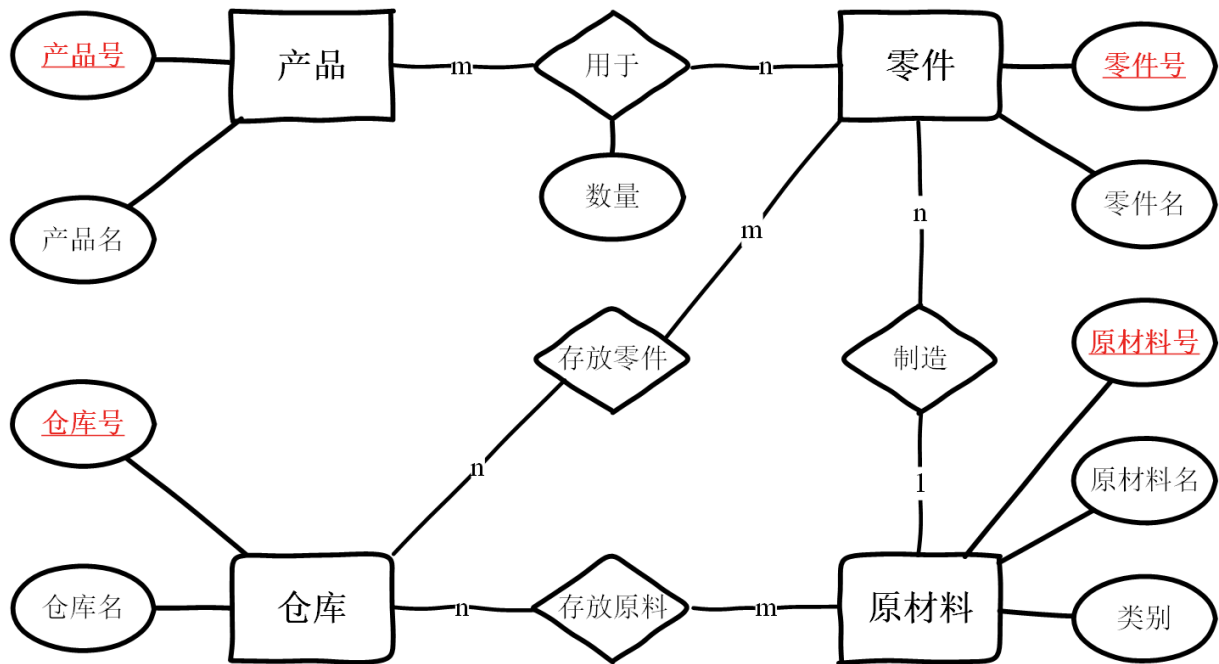


图 8: 工厂生产场景 E-R 图

关系模型转换如下:

- **产品** (产品号, 产品名)
主键: 产品号
外键: 无
- **原材料** (原材料号, 原材料名, 类别)
主键: 原材料号
外键: 无
- **仓库** (仓库号, 仓库名)
主键: 仓库号
外键: 无
- **零件** (零件号, 零件名)
主键: 零件号
外键: 无
- **零件-产品用于关系** (产品号, 零件号, 数量)
主键: (产品号, 零件号) (复合主键)
外键: 产品号, 零件号

在目前的情况下, 上述所有关系模式都已满足 BCNF, 无需进一步分解。